

asgn04: Constructors

Video

Watch [chapter 6: Magic Methods from the PHP: Object-oriented Programming video from InLearning](#).

Setup

- Create a new folder named `asgn04-constructors` in your `web250` folder.
- Create a file named `constructor.php` inside your `asgn04-constructors` folder.

Git

- Open VSCode and work in the terminal
- Navigate to your current working directory for this chapter.
- Complete the following git tasks
 - Stage and commit
 - Create a new branch named `asgn04-constructor`.
 - Switch to the new branch

Basic constructor

This exercise is based off video chapter 6.1, *Contstructor Method*

Create a class named `Bird` with two public properties

- `commonName`
- `latinName`

Note: when creating a constructor that requires the arguments that uses the `public` modifier. Create two new instances of the `Bird` class

- `commonName` = Robin
- `latinName` = Turdus migratorius
- `commonName` = Eastern Towhee
- `latinName` = Pipilo erythrophthalmus

Output

```
Common name: Robin
Latin name: Turdus migratorius
-----
Common name: Eastern Towhee
Latin name: Pipilo erythrophthalmus
```

Array of Arguments

This exercise is based off chapter 6.2, *Constructor Arguments*. This is a more flexible and readable way to create a constructor. You will use an array. Build on what you created in the previous exercise.

Code

- Create a new file named `constructor-arguments.php` in your `asgn04-constructors` folder.
- Create a class named `Bird` with two public properties
 - `commonName`
 - `latinName`
- Create a constructor that uses an array called `args[]`

```
public function __construct($args[]) {  
  
}
```

Use the null coalescing operator (shorthand ternary operator).

Create two new instances of the Bird class

- `commonName` = Acadian Flycatcher
- `latinName` = Turdus migratorius
- `commonName` = Eastern Towhee
- `latinName` = Pipilo erythrophthalmus

Output

```
Common name: Robin  
Latin name: Turdus migratorius  
-----  
Common name: Eastern Towhee  
Latin name: Pipilo erythrophthalmus
```

Git

Stage and commit your `asgn04-constructor` branch.

Autoload

This exercise is based off chapter 6.7, *Undefined Classes*

The purpose of autoload is to only load classes you use from a class library. This saves on memory if you have a large library of classes.

`bird.class.php`

Add the following

```
Class Bird
{
    public $commonName;
    public $latinName;

    public function __construct($args=[]) {
        $this->commonName = $args['commonName'] ?? NULL;
        $this->latinName = $args['latinName'] ?? NULL;
    }
}
```

Follow along with the video to create your `autoload.php` file.

It is easy to make mistakes when writing regular expressions, so here is the function and the registration.

```
function my_autoload($class) {
    if(preg_match('/\A[w+\Z/', $class)) {
        include 'classes/' . $class . '.class.php';
    }
}
spl_autoload_register('my_autoload');
```

You are required to create a new instance of a Acadian Flycatcher and echo its

Output Acadian Flycatcher

Git Once your code is working correctly

- Stage and commit your branch
- Switch to your `main` branch.
- Merge with your `asgn04-constructor` branch. *IMPORTANT*: Review (consult AI) if needed so you merge in the correct direction!
- Push your code to GitHub
- Copy your GitHub Address so you can submit it in Moodle.

Folder structure

When you are finished adding all of the files and folders, your folder tree should look like.



Submit Work

Paste your GitHub URL into the comments section of Moodle.